

INTERVIEW PREPARATION SERIES

Java Collections Framework

Scenario-Based Questions — 20 Real-World Scenarios

This document presents **20 real-world scenarios** spanning ArrayList performance, HashSet duplicate bugs, HashMap concurrency, TreeMap ordering, ConcurrentHashMap migration, CopyOnWriteArrayList patterns, PriorityQueue top-K, WeakHashMap caching, stream pipelines, and collection design for high-throughput systems. Each scenario simulates a real-world Java engineering situation.

List Scenarios

SCENARIO 01

A Java microservice processes 500 requests/second. Each request creates an `ArrayList` with an initial capacity of 10 but always adds 10,000 elements. Performance profiling shows 30% of CPU time in `Arrays.copyOf` (`ArrayList` resizing). Optimize this.

EXPECTED DEPTH

Problem: `ArrayList` starts with capacity 10 and doubles ~13 times to reach 10,000. Each resize copies the entire array.

```
// SLOW: 13 resizes for 10,000 elements
List<Item> items = new ArrayList<>(); // capacity 10
for (int i = 0; i < 10000; i++) items.add(process(i));

// FAST: pre-size to avoid resizing entirely
List<Item> items = new ArrayList<>(10000); // exact capacity
for (int i = 0; i < 10000; i++) items.add(process(i));
```

Result: zero resizes, 30% CPU reduction. **Always pre-size `ArrayList` when you know the expected size.** Also applies to `HashMap`: `new HashMap<>(expectedSize / 0.75 + 1)` avoids rehashing.

SCENARIO 02

A developer adds 50,000 elements to a `LinkedList` and then accesses elements by index in a loop: `for (int i = 0; i < list.size(); i++) list.get(i)`. The operation takes 45 seconds instead of milliseconds. Explain and fix.

EXPECTED DEPTH

Problem: `LinkedList.get(i)` is $O(n)$ — it traverses from the head (or tail) every time. In a loop, this is $O(n^2)$: $50,000 \times 25,000 = 1.25$ billion pointer traversals.

```
// O(n^2): NEVER index a LinkedList in a loop
for (int i = 0; i < list.size(); i++) {
    process(list.get(i)); // O(n) per call!
}

// O(n): use an iterator (or enhanced for loop)
for (Item item : list) {
    process(item); // O(1) per call via iterator next()
}

// BEST: use ArrayList if you need index access
List<Item> items = new ArrayList<>(linkedList); // convert once O(n)
for (int i = 0; i < items.size(); i++) {
    process(items.get(i)); // O(1) per call
}
```

Set & Uniqueness Scenarios

SCENARIO 03

A `HashSet<User>` contains duplicate users even though the users have the same name and email. The developer insists they overrode `equals()`. After investigation, they forgot to override `hashCode()`. Show the bug and fix.

EXPECTED DEPTH

```
// BUG: equals overridden, hashCode NOT overridden
class User {
    String name; String email;
    @Override public boolean equals(Object o) {
        if (!(o instanceof User)) return false;
        User u = (User) o;
```

```
        return name.equals(u.name) && email.equals(u.email);
    }
    // hashCode() inherited from Object – uses memory address!
}

User a = new User("Alice", "alice@co.com");
User b = new User("Alice", "alice@co.com");
a.equals(b); // true
Set<User> set = new HashSet<>();
set.add(a); set.add(b);
set.size(); // 2! (a and b have DIFFERENT hashCodes)
```

Fix: always override both:

```
@Override public int hashCode() {
    return Objects.hash(name, email); // MUST use same fields as equals
}
```

SCENARIO 04

A `TreeSet` of custom objects throws `ClassCastException` on the first `add()` call. The developer did not implement `Comparable`. Explain and provide two fixes.

EXPECTED DEPTH

Problem: `TreeSet` requires elements to be **Comparable** or a **Comparator** to be provided. Without either, it cannot determine ordering.

```
// CRASH: Product does not implement Comparable
TreeSet<Product> set = new TreeSet<>();
set.add(new Product("Widget", 9.99)); // ClassCastException!

// Fix 1: implement Comparable
class Product implements Comparable<Product> {
    public int compareTo(Product o) { return this.name.compareTo(o.name); }
}

// Fix 2: provide a Comparator
TreeSet<Product> set = new
TreeSet<>(Comparator.comparing(Product::getName));
set.add(new Product("Widget", 9.99)); // works!
```

Map Scenarios

SCENARIO 05

A web application uses a `HashMap` to cache user sessions. Under high concurrency (1000 concurrent requests), the application occasionally enters an infinite loop. In Java 7, this caused 100% CPU. Diagnose and fix.

EXPECTED DEPTH

Diagnosis: `HashMap` is **not thread-safe**. Concurrent `put()` operations during rehashing can create a circular linked list in a bucket, causing `get()` to loop forever.

Fix: use `ConcurrentHashMap`:

```
// BROKEN: unsynchronized HashMap with concurrent access
private Map<String, Session> sessions = new HashMap<>();

// FIXED: ConcurrentHashMap
private Map<String, Session> sessions = new ConcurrentHashMap<>();

// Atomic operations (no separate lock needed)
sessions.computeIfAbsent(sessionId, id -> createSession());
sessions.compute(sessionId, (id, session) -> session.refresh());
```

Note: Java 8+ `HashMap` uses trees instead of linked lists for long chains, which does not create infinite loops. But it still corrupts data under concurrent modification. **Always use `ConcurrentHashMap` for shared maps.**

SCENARIO 06

A reporting module groups 1 million orders by customer. The developer uses a `TreeMap` for the grouping. The operation takes 8 seconds. Switching to `HashMap` reduces it to 0.8 seconds. Explain the 10x difference.

EXPECTED DEPTH

Root cause: `TreeMap` uses a Red-Black tree. Every `put()` is $O(\log n)$ with tree rebalancing. `HashMap` uses hashing: `put()` is $O(1)$ average.

```
// SLOW: TreeMap O(n log n) for 1M inserts
Map<String, List<Order>> groups = new TreeMap<>();
```

```
for (Order o : orders) {
    groups.computeIfAbsent(o.getCustomerId(), k -> new
ArrayList<>()).add(o);
}

// FAST: HashMap O(n) for 1M inserts
Map<String, List<Order>> groups = new HashMap<>(estimatedCustomers);
for (Order o : orders) {
    groups.computeIfAbsent(o.getCustomerId(), k -> new
ArrayList<>()).add(o);
}
```

Rule: use **TreeMap** **only** when you need sorted keys. For grouping, counting, and lookups, **HashMap** is always the right choice.

SCENARIO 07

A developer stores mutable objects as `HashMap` keys. After modifying a key object, `map.get()` returns null even though the entry exists. The developer is confused. Explain the bug.

EXPECTED DEPTH

```
List<String> key = new ArrayList<>(List.of("a", "b"));
Map<List<String>, String> map = new HashMap<>();
map.put(key, "value");

// Mutate the key!
key.add("c"); // hashCode changes!

map.get(key); // null! The entry is in the OLD bucket
map.containsKey(key); // false!

// But the entry still exists:
map.size(); // 1
map.entrySet().forEach(System.out::println); // [a, b, c]=value
```

Why: `HashMap` computes the bucket from `hashCode()` at insertion time. After mutation, the key's `hashCode` changes, so `get()` looks in the wrong bucket.

Rule: **never use mutable objects as Map keys.** Use immutable types (`String`, `Integer`, record types) or ensure the fields used in `hashCode/equals` are never modified.

Concurrent & Performance Scenarios

SCENARIO 08

A multi-threaded application uses `Collections.synchronizedMap(new HashMap<>())` for a cache. Under load, throughput drops to single-threaded levels. Profiling shows threads waiting on the map's lock. Optimize.

EXPECTED DEPTH

Problem: `synchronizedMap` uses a **single lock** for ALL operations. 100 threads contend for one lock → effectively single-threaded.

```
// SLOW: global lock
Map<String, Data> cache = Collections.synchronizedMap(new HashMap<>());

// FAST: ConcurrentHashMap (lock striping)
Map<String, Data> cache = new ConcurrentHashMap<>();

// Even faster for read-heavy workloads: copy-on-write pattern
private volatile Map<String, Data> cache = Map.of();
void refresh() { cache = loadAll(); } // atomic reference swap
Data get(String key) { return cache.get(key); } // lock-free read
```

SCENARIO 09

A Java application tracks the last 1000 user actions using an `ArrayList`. When it exceeds 1000, the first element is removed: `list.remove(0)`. With millions of actions, this becomes very slow. Optimize.

EXPECTED DEPTH

Problem: `ArrayList.remove(0)` is **O(n)** — shifts all 999 remaining elements left.

```
// SLOW: O(n) per removal at index 0
if (actions.size() > 1000) actions.remove(0);

// FAST: ArrayDeque as bounded buffer O(1)
Deque<Action> actions = new ArrayDeque<>();
void addAction(Action a) {
    if (actions.size() >= 1000) actions.pollFirst(); // O(1)
    actions.addLast(a); // O(1)
}
```

ArrayDeque gives $O(1)$ at both ends. For a bounded-size FIFO, it is the correct data structure, not ArrayList.

SCENARIO 10

A report query returns 10 million rows from the database. The developer loads all into a `List<Map<String, Object>>`. The application throws `OutOfMemoryError`. Redesign the processing.

EXPECTED DEPTH

Problem: 10M rows $\times$ ~1KB per Map = ~10GB. Far exceeds JVM heap.

Fix 1 — Stream processing (process row by row):

```
// Use JDBC ResultSet streaming (don't load all into memory)
stmt.setFetchSize(1000); // fetch 1000 rows at a time
ResultSet rs = stmt.executeQuery(query);
while (rs.next()) {
    processRow(rs); // process one row at a time, constant memory
}
```

Fix 2 — Paginate:

```
// Process in 10,000-row batches
int offset = 0, batchSize = 10000;
while (true) {
    List<Order> batch = repo.findAll(PageRequest.of(offset/batchSize,
    batchSize));
    if (batch.isEmpty()) break;
    batch.forEach(this::process);
    offset += batchSize;
}
```

Rule: never load unbounded result sets into memory. Use streaming (ResultSet cursor), pagination, or database-side aggregation.

Advanced Collection Patterns

SCENARIO 11

Your application needs a collection that maintains elements in both insertion order AND sorted order, with $O(1)$ lookup. No

single standard collection does all three. Design a composite solution.

EXPECTED DEPTH

```
// Composite data structure: HashMap + TreeSet + insertion order list
class MultiIndexCollection<T> {
    private final Map<String, T> byKey = new LinkedHashMap<>(); //
    insertion order + O(1)
    private final TreeSet<T> sorted; // sorted order

    public MultiIndexCollection(Comparator<T> comp) {
        this.sorted = new TreeSet<>(comp);
    }
    public void add(String key, T item) {
        byKey.put(key, item);
        sorted.add(item);
    }
    public T getByKey(String key) { return byKey.get(key); } // O(1)
    public List<T> getInsertionOrder() { return new
    ArrayList<>(byKey.values()); }
    public NavigableSet<T> getSorted() { return
    Collections.unmodifiableNavigableSet(sorted); }
}
```

Trade-off: double memory usage. Each element is stored in two collections. This is a common pattern when you need multiple access patterns.

SCENARIO 12

A word frequency counter uses `HashMap<String, Integer>`. The code uses `containsKey` + `get` + `put` — three map lookups per word. Optimize to a single operation.

EXPECTED DEPTH

```
// SLOW: 3 lookups per word
for (String word : words) {
    if (map.containsKey(word)) {
        map.put(word, map.get(word) + 1);
    } else {
        map.put(word, 1);
    }
}

// BETTER: getOrDefault (2 lookups)
map.put(word, map.getOrDefault(word, 0) + 1);
```



```
// BEST: merge (1 atomic operation)
for (String word : words) {
    map.merge(word, 1, Integer::sum);
}
```

`merge()` performs a single lookup: if absent, stores the value; if present, applies the merge function. Also **atomic for ConcurrentHashMap**.

SCENARIO 13

An API returns user search results. The product team wants: remove duplicates (by email), maintain the order they were found, and limit to 50 results. Design the collection pipeline.

EXPECTED DEPTH

```
// HashSet: unique + insertion order
Set<User> seen = new HashSet<>();
List<User> results = new ArrayList<>();

for (User user : searchResults) {
    if (seen.add(user)) { // add returns false if duplicate
        results.add(user);
        if (results.size() >= 50) break;
    }
}

// Or with Streams:
List<User> results = searchResults.stream()
    .distinct() // uses equals/hashCode
    .limit(50)
    .collect(Collectors.toList());
```

SCENARIO 14

A developer needs to find the top 10 most frequent words in a 1GB text file. Loading the entire file into memory causes `OutOfMemoryError`. Design a streaming solution using collections.

EXPECTED DEPTH

```
// Stream the file line by line (constant memory)
```

```
Map<String, Long> counts = new HashMap<>();
try (BufferedReader reader = Files.newBufferedReader(path)) {
    String line;
    while ((line = reader.readLine()) != null) {
        for (String word : line.toLowerCase().split("\\W+")) {
            counts.merge(word, 1L, Long::sum);
        }
    }
}

// Get top 10 using PriorityQueue (min-heap of size 10)
PriorityQueue<Map.Entry<String, Long>> top10 = new PriorityQueue<>(
    Comparator.comparingLong(Map.Entry::getValue));
for (Map.Entry<String, Long> entry : counts.entrySet()) {
    top10.offer(entry);
    if (top10.size() > 10) top10.poll(); // remove smallest
}
```

Memory: $O(\text{unique words})$ for the map + $O(10)$ for the heap. The file is never fully loaded.

SCENARIO 15

A notification system must deliver messages to subscribers. Subscribers can be added/removed while messages are being delivered. Using a regular `ArrayList` causes `ConcurrentModificationException`. Fix this.

EXPECTED DEPTH

```
// BROKEN: ArrayList + concurrent modification
private List<Subscriber> subscribers = new ArrayList<>();
void notify(Message msg) {
    for (Subscriber s : subscribers) s.onMessage(msg); // CME if
    add/remove happens
}

// FIX: CopyOnWriteArrayList (ideal for few writes, many reads)
private List<Subscriber> subscribers = new CopyOnWriteArrayList<>();
void subscribe(Subscriber s) { subscribers.add(s); } // safe
void unsubscribe(Subscriber s) { subscribers.remove(s); } // safe
void notify(Message msg) {
    for (Subscriber s : subscribers) s.onMessage(msg); // iterates
    snapshot
}
```

CopyOnWriteArrayList is perfect for the observer pattern: subscribers change rarely (few writes), notifications happen frequently (many reads).

SCENARIO 16

Your cache uses `HashMap<UserId, UserProfile>`. Profiles are large (10KB each). With 1 million cached profiles, the JVM runs out of memory. You need the cache to automatically evict entries when memory is low. Design the solution.

EXPECTED DEPTH

```
// WeakHashMap: entries removed when key is no longer referenced
Map<UserId, UserProfile> cache = new WeakHashMap<>();
// Problem: UserId keys may be garbage-collected too aggressively

// Better: Caffeine cache (industry standard)
Cache<UserId, UserProfile> cache = Caffeine.newBuilder()
    .maximumSize(100_000)
    .expireAfterAccess(Duration.ofMinutes(30))
    .softValues()           // use SoftReferences for values
    .build();

// Or: LinkedHashMap LRU
Map<UserId, UserProfile> lru = new LinkedHashMap<>(16, 0.75f, true) {
    protected boolean removeEldestEntry(Map.Entry<UserId, UserProfile>
eldest) {
        return size() > 100_000; // evict when over limit
    }
};
lru = Collections.synchronizedMap(lru); // if thread safety needed
```

SCENARIO 17

Your team needs to implement a simple in-memory database index. Given a `List<Employee>`, build indexes on `department` and `salary` for fast lookups. Use standard Java collections.

EXPECTED DEPTH

```
class EmployeeIndex {
    // Index: department -> list of employees
    private Map<String, List<Employee>> byDept = new HashMap<>();
    // Index: salary range -> sorted set of employees
```

```

private TreeMap<Integer, List<Employee>> bySalary = new TreeMap<>();

void buildIndex(List<Employee> employees) {
    for (Employee e : employees) {
        byDept.computeIfAbsent(e.getDept(), k -> new
ArrayList<>()).add(e);
        bySalary.computeIfAbsent(e.getSalary(), k -> new
ArrayList<>()).add(e);
    }
}

List<Employee> findByDept(String dept) { return
byDept.getOrDefault(dept, List.of()); }

List<Employee> findBySalaryRange(int min, int max) {
    return bySalary.subMap(min, true, max, true).values().stream()
        .flatMap(List::stream).collect(Collectors.toList());
}
}

```

HashMap gives O(1) exact lookup. TreeMap gives O(log n) range queries via `subMap()`. This is how database indexes work conceptually.

SCENARIO 18

A multithreaded producer-consumer system uses a shared `ArrayList`. Producers add items; consumers remove from the front. Both operations fail under concurrency. Replace with the correct concurrent collection.

EXPECTED DEPTH

```

// BROKEN: ArrayList not thread-safe, remove(0) is O(n)
List<Task> queue = new ArrayList<>();
void produce(Task t) { queue.add(t); }
Task consume() { return queue.remove(0); }

// FIXED: BlockingQueue (thread-safe, blocks when empty)
BlockingQueue<Task> queue = new LinkedBlockingQueue<>(1000);
void produce(Task t) throws InterruptedException { queue.put(t); } //
blocks if full
Task consume() throws InterruptedException { return queue.take(); } //
blocks if empty

```

BlockingQueue is the standard producer-consumer data structure. It handles synchronization, blocking, and bounded capacity.

SCENARIO 19

Management wants a report showing: for each department, the employee with the highest salary and the employee with the lowest salary. Write a clean solution using Java collections and streams.

EXPECTED DEPTH

```
Map<String, Map<String, Employee>> report = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.teeing(
            Collectors.maxBy(Comparator.comparing(Employee::getSalary)),
            Collectors.minBy(Comparator.comparing(Employee::getSalary)),
            (max, min) -> Map.of(
                "highest", max.orElse(null),
                "lowest", min.orElse(null))
        )
    ));

// Or with traditional collections:
Map<String, Employee> highestByDept = new HashMap<>();
Map<String, Employee> lowestByDept = new HashMap<>();
for (Employee e : employees) {
    highestByDept.merge(e.getDept(), e,
        (a, b) -> a.getSalary() > b.getSalary() ? a : b);
    lowestByDept.merge(e.getDept(), e,
        (a, b) -> a.getSalary() < b.getSalary() ? a : b);
}
```

SCENARIO 20

A system processes events from multiple sources. Events must be deduplicated (same eventId = same event), ordered by timestamp, and the last 10,000 events must be kept in memory. No single standard collection satisfies all three requirements. Design a composite solution.

EXPECTED DEPTH

```
class EventStore {
    private final Map<String, Event> byId = new LinkedHashMap<>() {
        protected boolean removeEldestEntry(Map.Entry<String, Event> e) {
```

```
        return size() > 10_000;
    }
};
private final TreeMap<Instant, Event> byTime = new TreeMap<>();

synchronized void addEvent(Event event) {
    if (byId.containsKey(event.getId())) return; // deduplicate
    byId.put(event.getId(), event);
    byTime.put(event.getTimestamp(), event);
    // Trim time index to match
    while (byTime.size() > 10_000) byTime.pollFirstEntry();
}

Event getById(String id) { return byId.get(id); } // O(1)
List<Event> getTimeRange(Instant from, Instant to) {
    return new ArrayList<>(byTime.subMap(from, to).values()); // O(log
n + k)
}
}
```

Three requirements = three collections: LinkedHashMap (dedup + bounded), TreeMap (time-ordered range queries), synchronized methods (thread safety). This is a real-world pattern for event stores.